

# AI Tools & Usage Documentation

## Engineering Workflow, Tooling Stack, and Human-in-the-Loop Oversight Protocol

Document ID: AI-TOOL-v2.0 | Date: September 19, 2026 | Lifecycle Scope: Architecture, Implementation & Verification

### 1. AI Tooling Inventory & Utilization Matrix

During the architecture, implementation, and rigorous verification of the platform, foundation models and agentic developer tools were leveraged across five distinct engineering phases:

| AI Tool / Model              | Engineering Phase                     | Specific Engineering Contributions                                                                                                                                      |
|------------------------------|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Gemini 2.5 Flash             | Runtime AI Triage & Voice Engine      | Production runtime model for conversational voice intake, structured Pydantic JSON triage output, and clinical indicator extraction from verbatim patient quotes.       |
| Antigravity Agentic IDE      | Full-Stack Implementation & Debugging | Autonomous pair programming, terminal test execution ( <code>`pytest`</code> ), AST code editing, SQLite schema migrations, and Next.js frontend dashboard development. |
| Claude 3.5 Sonnet            | Domain Modeling & PRD Synthesis       | Formulation of FHIR-aligned relational schema entities, queue starvation mathematical derivations, and conservative consensus truth table definitions.                  |
| Synthetic Evaluation Harness | Automated Validation & Benchmarking   | Synthesized 30 clinically authentic dialogue scenarios (covering acute coronary events, sepsis, medication omission, and adversarial prompt injections).                |

### 2. Methodological AI Engineering Workflows

- Phase 1: Domain & Relational Schema Modeling:** Formulated structured entities connecting Hospital, User, Patient, Encounter, Discharge, Condition, Observation, Campaign, OutreachTask, CallRecord, EscalationRecord, and AuditEvent.
- Phase 2: Queue Prioritization Mathematics:** Derived continuous scoring functions balancing clinical risk with deadline pressure curves  $([1 - T_{\text{remain}}]/T_{\text{window}})^2$  and starvation aging factors  $(\alpha \cdot T_{\text{wait}})$ .
- Phase 3: Dual Independent Assessment & Consensus Arbiter:** Engineered the two-agent clinical consensus architecture where Assessment A (generative protocol reasoning) is checked by Assessment B (deterministic rule validator).
- Phase 4: Full-Stack Frontend Engineering:** Implemented Next.js 14+ operational dashboards featuring live persona switching, real-time queue monitors, and interactive dual assessment review drawers.

### 3. Human Verification & Oversight Protocol

To satisfy strict clinical software quality standards, all AI-generated designs, code, and schemas were governed by a four-tier human verification gate:

**Mandatory Four-Tier Verification Gate:**

1. **Concurrency & Locking Audit:** Manually verified SQLite WAL mode and transactional locking semantics under simulated parallel load.
2. **Clinical Grounding Review:** Verified that all emergency red-flag triggers (e.g. chest pressure, purulent discharge, weight gain > 3 lbs) strictly mirror ACC/AHA and CDC guidelines.
3. **Multi-Tenant Isolation Scrutiny:** Confirmed that every database query enforces tenant scoping, preventing cross-hospital data leakage.
4. **Automated Regression Suite:** Mandated 100% pass rate on pytest tests/ -v and python scripts/evaluate\_safety.py before code acceptance.

### 4. Engineering Velocity & Quality Impact

Pairing agentic coding assistants with deterministic verification gates yielded measurable productivity gains while enhancing safety:

- **5x Accelerated Delivery:** Complex multi-tenant schemas, priority queue algorithms, and RAG pipelines were implemented and verified in days rather than weeks.
- **Zero Regression Deployment:** Every bug encountered during UI testing (such as queue batch pagination or status string mismatches) was instantly localized, patched, and validated with regression tests.
- **Comprehensive Documentation:** Complete architectural specifications, PRD alignment matrices, and clinical safety benchmarks were continuously documented and version-controlled.